

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 629 — Analysis of Algorithms

Design techniques, rigorous analysis, and the boundary between tractable and intractable

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 1 (with CSCE 641 and CSCE 614)

Prerequisites: Discrete mathematics and proof technique (UC MATH 600); comfort with a programming language

Effort: 10–12 hours per week · 13 modules + 2 project weeks

Deliverable: An algorithms portfolio: every technique implemented, benchmarked against its predicted complexity, with the gap between theory and measurement explained

Course description

An algorithms course is usually sold as a catalogue of algorithms. That is the least durable part of it. The algorithms you will need in five years mostly have library implementations; what you will need is the judgement to recognise which problem you are holding, which technique it yields to, and whether the thing you want is possible at all.

So this course is organised around *techniques* and *arguments* rather than around a list. Divide and conquer, greedy exchange, dynamic programming, flow, randomisation, amortisation — and for each, the proof pattern that tells you whether your instinct is correct. A greedy algorithm that happens to work on your test cases is worth nothing; a greedy algorithm with an exchange argument is a theorem.

The second half turns to limits. Lower bounds tell you when to stop optimising. NP-completeness tells you when to stop looking for an exact algorithm and start designing an approximation. Knowing where those walls are is what separates an engineer who ships from one who spends a month on a problem that was proven impossible in 1972.

Throughout, you implement and measure. Asymptotic analysis is a model, and models have domains of validity. Part of this course is learning exactly where the model stops predicting your machine — a theme CSCE 614 takes up in earnest.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Choose and justify an algorithm design technique from the structure of a problem.
- 2 Solve recurrences by substitution, recursion tree, and the master theorem, and say when each applies.
- 3 Prove greedy algorithms correct with exchange arguments, and recognise when greed fails.
- 4 Design dynamic programs by identifying subproblem structure, and analyse their complexity.
- 5 Model problems as network flow or matching, and apply the max-flow min-cut theorem.
- 6 Prove a problem NP-complete by reduction from a known hard problem.
- 7 Design approximation algorithms with proven ratios when exactness is unavailable.
- 8 Explain where asymptotic analysis stops predicting real running time, and why.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What Analysis Means	What exactly are we measuring, and what is the measurement worth?
2	Divide and Conquer, and Recurrences	When does splitting a problem help, and how do you compute the cost?
3	Randomization and Expectation	Why does adding randomness make an algorithm more reliable?
4	Sorting and Selection: Limits and Workarounds	Why can't comparison sorting beat $n \log n$, and when can you?
5	Hashing and Amortized Analysis	How can a lookup be $O(1)$, and what is the catch?
6	Graph Search and Decomposition	What can you learn about a graph by walking it once?
7	Shortest Paths	Why does one algorithm not cover every shortest-path problem?
8	Minimum Spanning Trees and Union-Find	Why do two different greedy strategies both find the optimum?
9	Greedy Algorithms and Exchange Arguments	How do you know a greedy algorithm is correct rather than merely plausible?
10	Dynamic Programming	How do you avoid recomputing the same subproblem exponentially often?
11	Network Flow and Matching	Why does so much reduce to pushing water through pipes?
12	NP-Completeness and Reductions	How do you show a problem is hard, and what do you do then?
13	Approximation Algorithms	If you cannot have the optimum, what can you guarantee?

Projects

Project 1 — Measured complexity (after Module 5)

Implement eight algorithms from the first five modules and measure them against their predicted asymptotics. The deliverable is not the code — it is a written account of where measurement and theory diverged, and why.

Requirements

- Mergesort, quicksort (randomised), heapsort, insertion sort, binary search, randomised selection, a hash table with chaining, and a dynamic array with doubling.
- A harness that times each across at least six input sizes spanning three orders of magnitude, with repeated trials.
- Log-log plots with fitted slopes, compared against predicted exponents.
- At least three documented cases where the measured curve departs from the predicted one.

Done means

- Insertion sort beats mergesort below some crossover size. Report the size and explain it.
- Quicksort beats mergesort despite identical asymptotics. Explain in terms of memory, not operation counts.
- A hash table's performance degrades at a specific load factor. Show the curve and name the cause.

Project 2 — A hard problem, three ways (after Module 12)

Take one NP-hard problem and attack it three ways: an exact exponential algorithm, an approximation with a proven ratio, and a heuristic with no guarantee. Compare them honestly on real instances.

Requirements

- Choose from: vertex cover, TSP (metric), set cover, knapsack, graph colouring, or maximum independent set.
- An exact algorithm with branch and bound or DP over subsets, with the instance size at which it becomes unusable reported.
- An approximation algorithm with the ratio **proved**, not cited.
- A heuristic (local search, greedy with restarts, simulated annealing) with no guarantee.
- A written comparison on at least twenty instances: solution quality against running time.

Done means

- A plot of solution quality versus time for all three methods.
- A proof of your approximation ratio, written out.
- An instance where the heuristic beats the approximation algorithm, and one where it loses badly. Both explained.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
MIT 6.046J Design and Analysis of Algorithms (OCW, full video)	Primary lecture series. Graduate level, closely matched to this course's order. Watch the mapped lecture before each module.
MIT 6.006 Introduction to Algorithms, Spring 2020 (OCW, full video)	The undergraduate predecessor. Use it when a 6.046 lecture assumes background you do not have — particularly for hashing, graph search, and basic DP.
Jeff Erickson, Algorithms (free book)	The best free algorithms textbook in existence, and unusually good on <i>how to think</i> rather than only what is true. Primary written reference for this course.
Tim Roughgarden, Algorithms Illuminated (free videos)	A third voice, strong on intuition and on why each analysis is structured the way it is.
Competitive Programmer's Handbook (free)	Implementation-focused. Use for the practical details the theory courses skip, and pair it with the CSES problem set.
CSES Problem Set (free, auto-graded)	300 problems ordered by technique. The best free source of exercises with immediate feedback.

Tooling

- **One language you are fluent in.** C++ or Python. C++ if you want the constant factors to be visible; Python if you want to iterate faster. Do not learn a language and algorithms at once.

- **A benchmarking harness** you write in Module 01 and reuse all term: time a function across input sizes, write a CSV, plot it.
- **matplotlib** or equivalent. Every complexity claim in this course should be checked against a measured curve at least once.
- **A portfolio repository**, public, one directory per module.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.